

Tab 1

MVP Must-Have Outputs

For each team project, the system must generate:

1. Strategy recommendation
2. Robot architecture
3. Legal/rules checklist
4. Bill of materials
5. Budget breakdown
6. Supply availability tracking
7. Mechanism calculations
8. Motor/torque/RPM recommendations
9. Gear ratio suggestions
10. Starter CAD/3D concept
11. Step-by-step build guide
12. FTC SDK Java starter code
13. Autonomous mode starter code
14. Driver-controlled mode starter code
15. Hardware configuration guide
16. Testing checklist
17. Grant/sponsor email generator
18. AI chatbot for iteration
19. Driver control optimization from logs

That is the real MVP: not a tiny demo, but a complete first usable version.

Target Users

Primary Users

FTC students, especially:

- New teams
- Underserved teams
- Teams without experienced mentors

- Teams with limited budget
- Teams overwhelmed by kickoff

Secondary Users

- Coaches
- Mentors
- Parents helping with grants
- Team captains
- Programming leads
- Mechanical leads
- Outreach leads

User Skill Levels

The app should support three modes:

Beginner Mode

Explains everything simply, avoids jargon, provides step-by-step build guidance, and recommends conservative designs.

Intermediate Mode

Allows more customization, tradeoff comparisons, and mechanism alternatives.

Advanced Mode

Lets teams define strategies, constraints, scoring priorities, custom hardware, tuning values, and code architecture.

Main Product Flow

1. Team Onboarding

The user creates a team profile.

Required fields:

- Team name

- Team number
- Location
- Experience level
- Number of students
- Available mentors
- Build space limitations
- Tools available
- CAD experience
- Programming experience
- Existing parts inventory
- Budget
- Preferred supplier, starting with REV Robotics
- Competition timeline
- Robot priorities

Robot priorities could include:

- Low cost
- Easy to build
- Fast drivetrain
- Strong endgame
- Reliable autonomous
- Simple driver control
- Easy maintenance
- Maximum scoring potential

2. Season/Game Manual Input

The system ingests:

- Current season competition manual
- Field drawings
- scoring rules
- robot construction rules
- inspection checklist
- Q&A updates if available

- team updates
- official field resources

The app should allow:

- Upload PDF
- Use official FIRST URL
- Manual version tracking
- Update detection
- Rule citation extraction

The user should see:

- Manual version
- Last updated date
- Important robot constraints
- Game scoring summary
- Autonomous scoring opportunities
- TeleOp scoring opportunities
- Endgame scoring opportunities
- Major penalties
- Major robot construction limits

3. Team Strategy Input

The team can choose:

AI-Generated Strategy

The app analyzes the game and proposes a strategy.

Team-Provided Strategy

The team writes their own strategy.

Hybrid Strategy

The team writes a strategy, and the AI improves it.

Example strategy inputs:

- “We want a simple robot that scores consistently.”
- “We want a mecanum drivetrain and a fast intake.”
- “We want to focus on autonomous.”
- “We only have \$1,200.”
- “We are a rookie team and need something easy.”
- “We already own a REV Starter Bot kit.”
- “We want to be alliance-friendly, not necessarily top scorer.”

4. Robot Concept Generation

The system generates 3 robot concepts.

Each concept includes:

- Name
- Strategy fit
- Difficulty
- Estimated cost
- Build time
- Required tools
- Required parts
- Main mechanisms
- Pros
- Cons
- Risks
- Rule concerns
- Upgrade path

Example concepts:

Concept A: Simple Reliable Scorer

- Tank drivetrain
- Basic intake
- Single-stage arm
- Low cost
- Lower scoring ceiling

- Best for rookie teams

Concept B: Balanced Competitive Robot

- Mecanum drivetrain
- Active intake
- Linear slide
- Servo-controlled scoring mechanism
- Medium cost
- Medium complexity

Concept C: High Ceiling Robot

- Mecanum drivetrain
- Multi-stage lift
- Fast intake/outtake
- Vision-assisted autonomous
- Higher cost
- Higher risk

The team selects one concept or asks the AI to merge ideas.

Core Feature Specs

Feature 1: Rules-Aware RAG System

The RAG system is the brain that keeps the AI grounded in official documents.

Documents to Index

MVP should index:

- FTC Competition Manual
- FTC Docs
- FTC SDK docs/Javadocs
- REV Robotics docs
- REV product pages

- REV CAD resources if available
- Team uploaded PDFs
- Team uploaded strategy notes
- Team uploaded inventory spreadsheets

RAG Requirements

The system must:

- Chunk documents by section
- Preserve page numbers
- Preserve rule numbers
- Preserve document version
- Store source URL
- Store effective date
- Store update date
- Support semantic search
- Support keyword search
- Return citations
- Highlight conflicting or outdated sources

Rule Citation Format

Every rule-sensitive answer should include:

- Rule number
- Manual section
- Source document
- Version/date
- Short explanation
- Confidence level

Example:

Rule check: This design may be affected by R### in the Robot Construction Rules section. Verify with the latest official manual before inspection.

Critical Rule Behavior

The app should refuse to present rule-sensitive claims without citations.

Bad:

“This is definitely legal.”

Good:

“Based on the indexed manual, this appears likely legal under these cited rules, but the official manual and event inspectors remain authoritative.”

Feature 2: Parts Catalog / Supply Engine

The parts engine tracks what the robot needs.

MVP Supplier

Start with REV Robotics because FTC teams commonly use REV parts and the user specifically mentioned REV.

Later support:

- goBILDA
- AndyMark
- ServoCity
- McMaster-Carr
- custom 3D printed parts
- team inventory

Parts Data Model

Each part needs:

- Part name
- SKU
- Supplier
- Category
- Price
- Weight
- Dimensions
- Material

- CAD URL
- Product URL
- Stock status
- FTC legality status if known
- Compatible parts
- Required accessories
- Quantity
- Electrical requirements
- Mechanical properties
- Notes

Categories

- Motors
- Servos
- Hubs
- Sensors
- Structural parts
- Wheels
- Gears
- Sprockets
- Chain
- Belts
- Pulleys
- Shafts
- Bearings
- Brackets
- Fasteners
- Linear motion
- Batteries
- Wires
- Connectors
- Control system components
- 3D printable parts

Supply Output

The system should produce:

- Required BOM
- Optional BOM
- Spare parts list
- Already-owned parts
- Missing parts
- Estimated subtotal
- Shipping estimate placeholder
- Budget remaining
- Substitution suggestions
- “Buy first” priority ranking

Stock Caveat

Stock changes constantly, so live availability should be fetched from supplier pages or APIs if available. If not possible, the app should label availability as “last checked.”

Feature 3: Budget Optimizer

The budget system takes:

- Total team budget
- Existing inventory
- Must-have mechanisms
- Nice-to-have mechanisms
- Spare parts preference
- Competition deadline
- Supplier preference

It outputs:

- Total estimated robot cost
- Cost by subsystem
- Minimum viable build cost

- Recommended build cost
- Competition-ready build cost
- Upgrade path
- Parts to skip
- Parts to buy used/reuse
- Parts to 3D print
- Parts that need spares

Budget Modes

Ultra-Low Budget

Use existing parts, simpler mechanisms, fewer sensors.

Balanced Budget

Reliable robot with reasonable scoring potential.

Competitive Budget

Better drivetrain, better sensors, more spares, refined mechanisms.

Feature 4: Strategy Engine

The strategy engine turns the game manual into robot priorities.

Inputs

- Scoring table
- Match timing
- Autonomous opportunities
- Endgame scoring
- Penalties
- Robot constraints
- Alliance interactions
- Team skill level
- Budget
- Build timeline
- Driver skill
- Programming skill

Outputs

- Recommended scoring priorities
- What to ignore
- What to specialize in
- Autonomous strategy
- TeleOp strategy
- Endgame strategy
- Driver practice goals
- Alliance compatibility
- Risk/reward analysis

Example Output

For a rookie team:

- Build a reliable drivetrain first.
- Score one or two repeatable tasks instead of chasing every task.
- Avoid high-complexity endgame unless it is essential.
- Use a simple autonomous path that parks or scores reliably.
- Prioritize driver practice over mechanism complexity.

Feature 5: Physics and Math Verifier

This is one of the most important parts.

The app should not just say “use this motor.” It should show the math.

Calculations Needed

MVP should support:

- Torque required
- Motor torque available
- Gear ratio
- RPM
- Wheel speed
- Linear speed

- Acceleration estimate
- Lift force
- Arm torque
- Stall safety margin
- Current draw estimate
- Battery load estimate
- Center of gravity estimate
- Tipping risk estimate
- Chain/belt speed
- Pulley ratio
- Servo torque margin
- Intake roller speed
- Linear slide load

Required Formulas

Wheel Speed

```

wheel_rpm = motor_rpm / gear_ratio
wheel_circumference = pi * wheel_diameter
linear_speed = wheel_rpm * wheel_circumference

```

Torque at Wheel

```

wheel_torque = motor_torque * gear_ratio * efficiency

```

Force at Wheel

```

force = wheel_torque / wheel_radius

```

Arm Torque

```

required_torque = load_weight * arm_length

```

With safety factor:

```

recommended_torque = required_torque * safety_factor

```

Lift Force

```

required_force = mass * gravity

```

Pulley Lift Torque

```

required_torque = required_force * pulley_radius

```

Safety Margin

```

safety_margin = available_torque / required_torque

```

Math Output Requirements

For every mechanism, the app should show:

- Assumptions

- Formula
- Inputs
- Calculation
- Result
- Safety factor
- Recommendation
- Warning if margin is too low

Example:

The lift requires 1.8 Nm of torque. The selected motor and gear ratio provide an estimated 3.1 Nm after efficiency loss, giving a safety margin of 1.72x. This is acceptable but not generous; consider a higher gear ratio if the lift binds.

Feature 6: CAD / Blueprint Generator

For MVP, do not promise perfect manufacturing-ready CAD. Instead, generate structured CAD concepts and exportable starter files.

MVP CAD Output

The system should generate:

- Subsystem layout
- Robot dimensions
- Drivetrain layout
- Mechanism placement
- Mounting points
- 2D blueprint views
- Exploded assembly diagrams
- Basic 3D model concept
- STEP/Onshape-compatible part references where possible
- CAD assembly instructions

Recommended MVP Approach

Use a combination of:

- Parametric CAD generation
- Existing REV CAD part files
- OpenSCAD or CadQuery for generated geometry
- Three.js for browser previews
- Optional Onshape API integration later

CAD Stack

Recommended:

- CadQuery for parametric mechanical models
- OpenCascade through CadQuery
- Three.js for web 3D preview
- React Three Fiber if using React
- glTF/GLB for browser display
- STEP export for CAD compatibility
- STL export for 3D printable parts
- Onshape API as a later premium/advanced integration

Blueprint Views

Each design should include:

- Top view
- Front view
- Side view
- Isometric view
- Exploded view
- Wiring view
- Subsystem closeups

CAD Reality Check

AI-generated CAD must be marked as:

“Conceptual CAD starter. Verify dimensions, clearances, fasteners, and legality before manufacturing.”

Feature 7: Build Instructions

The build guide should feel like LEGO-style instructions.

Build Guide Output

Each step includes:

- Step number
- Title
- Parts needed
- Tools needed
- Estimated time
- Image/diagram
- Instructions
- Safety warning if needed
- Checkpoint
- Common mistake
- Test before continuing

Build Phases

1. Prepare parts
2. Build drivetrain
3. Mount control hub
4. Wire drivetrain
5. Test drivetrain
6. Build intake
7. Build scoring mechanism
8. Add sensors
9. Cable management
10. Upload code
11. Configure hardware map
12. Test subsystems
13. Tune autonomous
14. Driver practice
15. Inspection checklist

Build Verification

Each subsystem should include tests.

Example drivetrain test:

- Robot moves forward when left stick moves forward
 - Robot strafes correctly if mecanum
 - Motors are not reversed incorrectly
 - No wires drag
 - Battery is secure
 - Wheels spin freely
 - Robot can be disabled quickly
-

Feature 8: Code Generator

The code generator should produce FTC SDK Java code.

FTC Docs describe the SDK as the software used for developing and executing code on an FTC robot, including Driver Station, Robot Controller, Blocks, OnBot Java, Android Studio project, Javadocs, and season-specific assets.

MVP Code Targets

Start with:

- FTC SDK Java
- Android Studio project structure
- TeamCode classes
- LinearOpMode starter code
- TeleOp OpMode
- Autonomous OpMode
- Hardware mapping
- Subsystem classes
- Basic telemetry
- Safety checks

Later Targets

- Blocks code
- OnBot Java export
- Road Runner integration
- Pedro Pathing integration
- AprilTag vision
- OpenCV pipelines
- Dashboard tuning

Code Output

The app should generate:

- `RobotHardware.java`
- `DriveSubsystem.java`
- `LiftSubsystem.java`
- `IntakeSubsystem.java`
- `TeleOpMain.java`
- `AutoMain.java`
- `Constants.java`
- `README.md`
- Hardware configuration checklist

Code Style

Use:

- Clear class names
- Beginner-readable comments
- FTC SDK-compatible imports
- No fake libraries unless selected
- Hardware names matching the generated configuration
- Safe motor power limits
- Telemetry for debugging

Example Hardware Map

The generated code should match configuration names like:

```
leftFront = hardwareMap.get(DcMotor.class, "left_front");
rightFront = hardwareMap.get(DcMotor.class, "right_front");
liftMotor = hardwareMap.get(DcMotor.class, "lift_motor");
intakeServo = hardwareMap.get(Servo.class, "intake_servo");
```

REV documentation notes that configuration names are case-sensitive, so the app should enforce consistent naming.

Code Safety Features

Generated code should include:

- Motor power clipping
 - Emergency stop behavior
 - Servo range limits
 - Encoder reset options
 - Telemetry status
 - Hardware initialization errors
 - Battery voltage warnings if available
 - Comments explaining where students should tune constants
-

Feature 9: Autonomous Assistant

The autonomous assistant should help teams create paths and scoring routines.

Inputs

- Starting position
- Alliance color
- Desired scoring action
- Robot drivetrain type
- Sensors available
- Vision available
- Reliability preference

Outputs

- Autonomous strategy
- Path sequence

- Pseudocode
- Java starter code
- Tuning constants
- Testing plan

MVP Autonomous Types

- Drive forward/park
- Score preload
- Turn to heading
- Encoder-based movement
- Time-based fallback
- Basic sensor-based stop
- Optional AprilTag detection if supported

Autonomous Warnings

The app should say when a plan depends on tuning:

- Wheel diameter
- Encoder ticks per revolution
- Gear ratio
- Track width
- Battery voltage
- Floor traction
- Robot weight

Feature 10: Driver-Control Optimization

This is a strong differentiator.

The app should collect driver logs and suggest better controls.

Logged Events

- Button presses
- Stick movement

- Trigger values
- Mode toggles
- Action timestamps
- Match phase
- Robot state
- Driver errors
- Failed actions
- Repeated actions
- Time between actions

Driver Optimization Output

The app suggests:

- Better button mapping
- Macros
- Toggle vs hold controls
- Deadzone tuning
- Drive speed scaling
- Slow mode button
- Presets for lift positions
- Intake/outtake automation
- Controller layout for driver 1 and driver 2

Example Suggestion

The driver presses A then Right Trigger together 37 times per match. Consider binding this as a single “score” macro on Right Bumper.

MVP Implementation

FTC SDK code can log gamepad input to telemetry or a local file-like format. The app can accept uploaded logs as CSV/JSON.

Later, build direct syncing from Robot Controller logs.

Feature 11: AI Chatbot

The chatbot is the main interaction layer.

Chatbot Modes

- Rules assistant
- Strategy mentor
- Mechanical design assistant
- Code assistant
- CAD assistant
- Budget assistant
- Grant/outreach assistant
- Driver optimization assistant
- Debugging assistant

Chatbot Requirements

The chatbot must know the current project context:

- Budget
- Selected robot concept
- Parts list
- Code generated
- Build progress
- Known issues
- Team skill level
- Competition timeline

Chatbot Example Questions

- "Is this intake legal?"
- "Can we make this cheaper?"
- "Why does our lift stall?"
- "Generate code for our slide."
- "What gear ratio should we use?"
- "How do we wire this sensor?"
- "Write an email asking a local company for \$500."
- "Our driver keeps missing this button. What should we change?"

Feature 12: Grant and Sponsor Assistant

This feature supports team sustainability.

Inputs

- Team name
- Location
- Mission
- Budget need
- Student demographics
- Past achievements
- Sponsor type
- Donation goal
- Contact name
- Company name

Outputs

- Sponsor email
- Grant application draft
- Budget justification
- One-page sponsorship packet text
- Follow-up email
- Thank-you email
- Donation tiers
- Outreach tracker

Sponsor CRM MVP

Track:

- Organization
- Contact name
- Email
- Status

- Amount requested
 - Amount received
 - Follow-up date
 - Notes
-

Technical Architecture

Recommended MVP Stack

Frontend

Use:

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Lucide icons
- React Hook Form
- Zod
- TanStack Query
- React Three Fiber / Three.js

Why:

- Fast web app development
- Strong form handling
- Good dashboard UX
- Type safety
- Easy 3D preview support

Backend

Use:

- Python FastAPI

Why:

- Best ecosystem for AI, document processing, CAD generation, math, and agents
- Easy async APIs
- Easy integration with vector DBs and ML tools

Agent Orchestration

Use:

- LangGraph for multi-agent workflows
- LlamaIndex or LangChain for RAG utilities
- OpenAI / Anthropic / Google model adapter layer
- Instructor or Pydantic structured outputs

The key is model portability. Do not hardcode the product to one model.

Database

Use:

- PostgreSQL
- pgvector
- Prisma or Drizzle for TypeScript app data
- SQLAlchemy or SQLModel for Python services if needed

Store:

- Users
- Teams
- Projects
- Parts
- BOMs
- Documents
- Embeddings
- Designs
- Code artifacts
- Chat history
- Build steps

- Driver logs
- Sponsor contacts

File Storage

Use:

- S3-compatible storage

Options:

- AWS S3
- Cloudflare R2
- Supabase Storage
- MinIO for local dev

Store:

- PDFs
- CAD files
- generated images
- code zips
- logs
- exports
- build guide PDFs

Queue / Background Jobs

Use:

- Redis
- BullMQ if Node-based jobs
- Celery or RQ if Python-based jobs
- Temporal later for more reliable complex workflows

MVP recommendation:

- Redis + Celery for Python-heavy AI/CAD jobs

Vector Search

Use:

- pgvector for MVP

Later:

- Qdrant
- Weaviate
- Pinecone

pgvector is enough for MVP and keeps infrastructure simpler.

Authentication

Use:

- Clerk
- or Auth.js
- or Supabase Auth

MVP recommendation:

- Clerk for speed
- Supabase Auth if using Supabase stack

Payments

Optional for MVP, but if needed:

- Stripe

Deployment

MVP deployment:

- Frontend: Vercel
- Backend: Render, Fly.io, Railway, or AWS ECS
- DB: Supabase Postgres or Neon
- Redis: Upstash
- Storage: Cloudflare R2 or S3

More serious production:

- AWS ECS/Fargate
 - RDS Postgres
 - ElastiCache Redis
 - S3
 - CloudFront
-

Agent System Design

The product should use multiple specialized agents rather than one giant prompt.

Agents

1. Intake Agent

Collects team constraints and asks clarifying questions.

2. Rules Agent

Searches official documents and checks legality.

3. Strategy Agent

Analyzes game scoring and recommends priorities.

4. Mechanical Design Agent

Creates subsystem architecture.

5. Parts Agent

Builds BOM from catalog and budget.

6. Physics Agent

Calculates torque, RPM, weight, and safety factors.

7. CAD Agent

Generates parametric CAD starter files and diagrams.

8. Code Agent

Generates FTC SDK Java code.

9. Build Guide Agent

Creates step-by-step assembly instructions.

10. Driver Optimization Agent

Analyzes pilot logs and suggests control improvements.

11. Grant Agent

Creates sponsor emails and grant drafts.

12. Review Agent

Checks outputs for contradictions, missing citations, risky assumptions, and impossible designs.

Agent Flow

Team Inputs

- > Intake Agent
- > Rules Agent
- > Strategy Agent
- > Mechanical Design Agent
- > Parts Agent
- > Physics Agent
- > CAD Agent
- > Code Agent
- > Build Guide Agent
- > Review Agent
- > Final Team Plan

Every major output should pass through the Review Agent.

Prompt Architecture

System Prompt Principles

The AI should be instructed to:

- Prioritize FTC legality
- Cite rules

- Respect budget
- Prefer simpler mechanisms for beginners
- Avoid unsupported claims
- Show math
- Be honest about uncertainty
- Recommend mentor review
- Generate editable outputs
- Preserve student learning

Project Context Object

Every agent should receive structured context:

```
{
  "team": {
    "name": "Example Robotics",
    "number": "12345",
    "experience": "rookie",
    "budget": 1500,
    "tools": ["basic hand tools", "3D printer"],
    "programmingExperience": "beginner"
  },
  "season": {
    "name": "DECODE",
    "manualVersion": "TU32",
    "manualUpdated": "2026-04-16"
  },
  "strategy": {
    "mode": "hybrid",
    "priorities": ["reliable scoring", "simple autonomous"]
  },
  "constraints": {
    "supplier": "REV",
    "maxBuildTimeWeeks": 6,
    "mustUseExistingParts": true
  }
}
```

Data Model

Tables

users

- id
- email
- name
- role
- created_at

teams

- id
- name
- number
- location
- experience_level
- budget
- created_by

projects

- id
- team_id
- season
- manual_version
- status
- selected_strategy
- selected_design

documents

- id
- project_id
- title
- source_url
- type
- version
- uploaded_at

- checksum

document_chunks

- id
- document_id
- section
- page
- rule_number
- text
- embedding

parts

- id
- supplier
- sku
- name
- category
- price
- weight
- dimensions
- product_url
- cad_url
- metadata

inventories

- id
- team_id
- part_id
- quantity
- condition

boms

- id

- project_id
- name
- total_cost

bom_items

- id
- bom_id
- part_id
- quantity
- required
- subsystem

designs

- id
- project_id
- name
- difficulty
- estimated_cost
- design_json

mechanisms

- id
- design_id
- type
- name
- inputs_json
- calculations_json

generated_code

- id
- project_id
- file_name
- language

- content
- created_at

build_steps

- id
- project_id
- step_number
- title
- instructions
- parts_json
- tools_json

driver_logs

- id
- project_id
- uploaded_by
- log_file_url
- parsed_json
- created_at

sponsor_contacts

- id
 - team_id
 - organization
 - contact_name
 - email
 - status
 - amount_requested
 - amount_received
 - next_followup
-

API Endpoints

Project APIs

```
POST /api/projects
GET /api/projects/:id
PATCH /api/projects/:id
DELETE /api/projects/:id
```

Document APIs

```
POST /api/projects/:id/documents/upload
POST /api/projects/:id/documents/ingest-url
GET /api/projects/:id/documents
GET /api/projects/:id/rules/search
```

Design APIs

```
POST /api/projects/:id/generate-strategies
POST /api/projects/:id/generate-designs
POST /api/projects/:id/select-design
GET /api/projects/:id/designs
```

BOM APIs

```
POST /api/projects/:id/generate-bom
GET /api/projects/:id/bom
PATCH /api/projects/:id/bom/items/:itemId
```

Physics APIs

```
POST /api/projects/:id/calculate/mechanism
GET /api/projects/:id/calculations
```

CAD APIs

```
POST /api/projects/:id/generate-cad
GET /api/projects/:id/cad
GET /api/projects/:id/cad/export?format=step
```

Code APIs

```
POST /api/projects/:id/generate-code
GET /api/projects/:id/code
GET /api/projects/:id/code/export.zip
```

Build Guide APIs

```
POST /api/projects/:id/generate-build-guide
GET /api/projects/:id/build-guide
GET /api/projects/:id/build-guide.pdf
```

Chat APIs

```
POST /api/projects/:id/chat
GET /api/projects/:id/chat/history
```

Driver Log APIs

```
POST /api/projects/:id/driver-logs/upload
POST /api/projects/:id/driver-logs/analyze
GET /api/projects/:id/driver-optimization
```

Grant APIs

```
POST /api/teams/:id/sponsor-email
POST /api/teams/:id/grant-draft
GET /api/teams/:id/sponsor-contacts
PATCH /api/teams/:id/sponsor-contacts/:contactId
```

Frontend Pages

Dashboard

Shows:

- Active team
- Current project
- Budget status
- Build progress
- Latest AI recommendations
- Open warnings
- Next steps

New Project Wizard

Steps:

1. Team info
2. Budget
3. Inventory
4. Game manual
5. Strategy
6. Constraints
7. Generate

Strategy Page

Shows:

- Scoring breakdown
- AI recommended strategy
- Team strategy editor
- Risk/reward comparison
- Selected strategy

Robot Concepts Page

Shows 3 generated concepts with:

- Cost
- Difficulty
- Scoring potential
- Reliability
- Timeline
- Rule risks
- Select/merge buttons

BOM Page

Shows:

- Parts table
- Supplier links
- Cost
- Quantity

- Subsystem
- In inventory
- Need to buy
- Substitutions

Physics Page

Shows:

- Mechanism cards
- Torque calculations
- RPM calculations
- Safety margins
- Warnings
- Formula explanations

CAD Page

Shows:

- 3D preview
- Top/front/side views
- Export buttons
- Part placement
- Notes

Code Page

Shows:

- File tree
- Java code editor
- Download ZIP
- Hardware map
- Configuration checklist

Build Guide Page

Shows:

- Step-by-step instructions
- Parts per step
- Diagrams
- Checkpoints
- Mark step complete

Chat Page

Shows:

- Project-aware chatbot
- Source citations
- Suggested actions
- “Apply to plan” button

Driver Logs Page

Shows:

- Upload logs
- Control heatmap
- Button usage
- Suggested remaps
- Macro suggestions

Grants Page

Shows:

- Sponsor CRM
- Email generator
- Grant draft generator
- Follow-up reminders

MVP UX Requirements

The app should feel like a serious engineering tool, not a toy chatbot.

Visual Style

Use:

- Clean dashboard layout
- Dense but readable tables
- Engineering-style panels
- Tabs
- Diagrams
- Source citations
- Warning badges
- Cost indicators
- Progress checklists

Avoid:

- Marketing landing page as the main screen
- Giant hero sections
- Overly cute UI
- Fake certainty
- Uncited rule claims

Main Navigation

- Dashboard
 - Strategy
 - Design
 - BOM
 - Physics
 - CAD
 - Code
 - Build Guide
 - Chat
 - Driver Logs
 - Grants
-

MVP Acceptance Criteria

The MVP is successful when a team can:

1. Create a team profile.
 2. Upload or select the current FTC manual.
 3. Enter a budget and constraints.
 4. Generate 3 robot concepts.
 5. Select one robot concept.
 6. Receive a full BOM with estimated cost.
 7. See rules citations for legal concerns.
 8. See torque/RPM calculations for major mechanisms.
 9. View a basic CAD-style robot layout.
 10. Download starter FTC Java code.
 11. Follow build steps.
 12. Ask the chatbot project-specific questions.
 13. Generate a sponsor email.
 14. Upload driver logs and get control suggestions.
-

Engineering Milestones

Phase 1: Foundation

- Auth
- Team/project setup
- Database schema
- File upload
- Manual ingestion
- Basic RAG search

Phase 2: AI Planning

- Strategy generator
- Robot concept generator
- Rules citation system

- Review agent

Phase 3: BOM and Budget

- REV parts ingestion
- BOM generator
- Budget optimizer
- Inventory tracking

Phase 4: Physics

- Mechanism calculators
- Safety factor reports
- Math explanation UI

Phase 5: Code

- FTC SDK Java templates
- Hardware map generator
- TeleOp generator
- Autonomous starter generator
- Export ZIP

Phase 6: CAD and Build Guide

- Parametric layout generator
- Three.js preview
- Build instruction generator
- PDF export

Phase 7: Driver Logs and Grants

- Log upload
- Controller usage analysis
- Sponsor CRM
- Email/grant drafts

Biggest Risks

1. Legal/Rules Hallucination

Mitigation:

- RAG citations required
- No uncited legality claims
- Manual version tracking
- “Verify with official manual” warnings

2. Bad Mechanical Advice

Mitigation:

- Physics calculators
- Safety margins
- conservative defaults
- mentor review warnings
- design risk ratings

3. CAD Overpromising

Mitigation:

- Label CAD as starter/conceptual
- Use parametric constraints
- require student verification

4. Code That Does Not Compile

Mitigation:

- Use FTC SDK templates
- generate unit-style static checks where possible
- test generated code in a sample FTC project
- avoid unsupported libraries by default

5. Outdated Parts Prices or Stock

Mitigation:

- show “last checked”
- allow manual price overrides
- supplier URL citations
- inventory override

6. Students Using It as a Replacement

Mitigation:

- explain reasoning
- show math
- ask students to make decisions
- include reflection prompts
- require design review checkpoints

Suggested Tech Stack Summary

Use this stack:

- Frontend: Next.js, React, TypeScript, Tailwind, shadcn/ui, Lucide
- 3D/CAD Viewer: Three.js, React Three Fiber, glTF/GLB
- Backend: FastAPI, Python
- Database: PostgreSQL + pgvector
- Queue: Redis + Celery
- Storage: S3 or Cloudflare R2
- RAG: LlamaIndex or LangChain
- Agents: LangGraph
- Structured AI Output: Pydantic / Instructor
- CAD Generation: CadQuery, OpenCascade, STEP/STL export
- Code Generation: FTC SDK Java templates
- Auth: Clerk or Supabase Auth
- Deployment: Vercel frontend, Render/Fly/AWS backend, Supabase/Neon Postgres

- Observability: Sentry, OpenTelemetry, PostHog
 - Testing: Playwright, Vitest, Pytest
 - CI/CD: GitHub Actions
-

Final MVP Positioning

The MVP should be positioned as:

“An AI engineering co-pilot for FTC teams that turns the official game manual, team budget, strategy, and constraints into a complete robot planning package: legal checklist, BOM, cost estimate, physics-backed mechanism plan, starter CAD, FTC code, build guide, grant support, and driver optimization.”

The strongest version of this idea is not just “ChatGPT for FTC.”

It is a full project workspace where every AI answer connects back to:

- official rules,
- real parts,
- team budget,
- math,
- CAD,
- code,
- and build instructions.

That is what makes it actually useful.

Originals

Yes. This is a strong idea, but the key is to position it as an **“FTC design copilot”**, not an “AI builds the whole robot for you” magic box. The best version helps teams move from messy constraints to a legal, budgeted, explainable starting design.

****Current Landscape****

FIRST already has official season materials and, notably, a **“Competition Manual AI Chatbot”** for the 2025-2026 DECODE season, so your tool should not just be “manual chatbot 2.0.” It needs to combine the manual with parts, CAD, budgeting, physics, code generation, strategy, and team operations. The current FTC game materials are hosted by FIRST, with the DECODE manual listed as Version TU22 updated February 5, 2026: [FIRST Game & Season Materials](<https://ftc-resources.firstinspires.org/ftc/game?hsLang=en>).

REV is a realistic starting vendor because its FTC Starter Kit V3.1 page exposes structured-ish product info, price, kit contents, documentation, and CAD links. Example: the FTC Starter Kit V3.1 is listed at `\$695.00` and includes kit contents plus CAD/build guide resources: [REV FTC Starter Kit V3.1](<https://www.revrobotics.com/rev-45-3529/>).

FTC programming has an official SDK workflow. FIRST lists the FTC SDK and required software versions, including FTC SDK Version 11.0 updated September 18, 2025 on the team resources page: [FIRST Team Resources](<https://ftc-resources.firstinspires.org/ftc/team>). FIRST also points teams to the official programming resources and GitHub releases: [FIRST Programming Resources](<https://www.firstinspires.org/resources/library/ffc/programming-resources>).

****Best Product Shape****

Build it as five connected modules:

1. ****Rules + Strategy RAG****

Upload or select the current game manual, team constraints, budget, existing inventory, experience level, and scoring goals. The system extracts robot size rules, legal parts constraints, match objectives, inspection risks, and scoring tradeoffs.

2. ****Parts + Budget Planner****

Scrape or ingest REV product pages, BOM PDFs, CAD links, prices, availability, and kit contents. Output:

- required parts
- optional upgrades
- reused inventory
- estimated cost
- missing items
- legal/rules notes
- source links

3. ****Design Generator****

Instead of promising full perfect CAD immediately, start with templated robot archetypes:

- tank drive starter bot
- mecanum drive
- linear slide scorer
- intake + transfer
- arm/claw
- endgame attachment if season-relevant

Then the AI adapts dimensions, parts, and mechanisms. CAD generation can begin with Onshape/Fusion-compatible STEP assemblies from known parts, then later add AI-generated geometry.

4. ****Physics Verifier****

This is where the product becomes credible. For every proposed mechanism, calculate:

- motor free speed and loaded speed
- gear ratio
- wheel RPM
- linear slide torque
- arm torque
- estimated acceleration
- stall current risk
- battery/load assumptions
- safety factor

The assistant should show formulas, assumptions, and warnings. Example: "This arm requires $X \text{ N}\cdot\text{m}$ at the joint; selected motor+gearbox provides $Y \text{ N}\cdot\text{m}$ theoretical stall, but usable torque should be derated."

5. ****Code + Driver Optimization****

Generate FTC Java code using the official SDK structure. For advanced autonomous, integrate common FTC libraries like Road Runner, which is a motion planning library for FTC: [Road Runner intro](<https://learnroadrunner.com/introduction>). For teleop, collect pilot logs:

- button presses
- joystick movement
- mode switches
- cycle time
- failed actions
- driver/operator conflicts

Then recommend control remaps, macros, toggle/hold changes, speed scaling, and automation assists.

****Killer MVP****

Do not start with "full CAD robot from scratch." Start with this:

> “Upload the game manual, enter budget/inventory/strategy, and get a legal FTC robot concept with REV parts, cost, physics proof, build steps, and starter Java code.”

MVP output should include:

- **Strategy summary:** what scoring tasks the robot prioritizes.
- **Robot architecture:** drivetrain, manipulator, sensors, control system.
- **BOM:** REV SKU, quantity, price, source link.
- **Build guide:** step-by-step, sectioned like LEGO instructions.
- **Physics sheet:** gear ratio, torque, RPM, safety factor.
- **FTC Java code:** teleop, hardware map, autonomous skeleton.
- **Risk report:** legal concerns, hard-to-build parts, budget pressure, inspection notes.

****Data Sources To Use****

Use official and semi-official sources first:

- FIRST game manuals and season docs: [FTC Game Materials](<https://ftc-resources.firstinspires.org/ftc/game?hsLang=en>)
- FIRST programming resources: [Programming Resources](<https://www.firstinspires.org/resources/library/ffc/programming-resources>)
- FIRST team management, grants, budgeting, fundraising: [Team Management Resources](<https://www.firstinspires.org/resources/library/ftc/team-management-resources>)
- FIRST grant listings: [Team Grant Opportunities](<https://www.firstinspires.org/robotics/team-grants>)
- REV product pages, CAD, guides, BOMs: [REV FTC Starter Kit](<https://www.revrobotics.com/rev-45-3529/>)
- REV documentation/build guides: [REV Docs](<https://docs.revrobotics.com/>)
- FTC libraries: [FTCLib](<https://docs.ftclib.org/>) and [Road Runner](<https://learnroadrunner.com/introduction>)

****Important Caveats****

The AI should never claim “this robot is guaranteed legal.” It should say “based on the cited manual sections, this appears legal, but confirm with the current manual, Q&A forum, and inspectors.”

Also, AI-generated CAD and code need verification. The tool should include simulation/checking steps, not just output files. This is especially true for torque, stall current, servo loading, and autonomous paths.

****Grant/Email Feature****

This is a good adjacent feature. FIRST already has official grant and fundraising resources, so your app can ask for:

- team number
- school/nonprofit status

- location
- rookie/veteran
- budget need
- sponsor category
- outreach story
- requested amount

Then it generates:

- grant search results
- sponsor email drafts
- budget justification
- thank-you templates
- follow-up reminders

****Differentiator****

The strongest pitch is:

> “An FTC copilot that turns rules, budget, inventory, and strategy into a buildable robot plan with sourced parts, physics-backed mechanism choices, starter code, and team operations support.”

That is much better than “AI robot designer,” because it sounds useful, safe, and believable.

I’d build the first prototype around REV-only parts, one season manual, three robot archetypes, and Java SDK code generation. Once that works, expand to goBILDA, AndyMark, Onshape integration, pilot telemetry, and grant automation.

Wait, okay, three, two... We need to make an FTC like a helper which will be able to get your like game manual, get your budget, get your like restrictions, what you want, specifications, your end goal, which is basically from the um um FTC like company, whatever, and it'll... Rev. That's the catalog, bro, that's not the damn thing that Milhouse the FTC. Okay. So, now which will be put into a prompt and the prompt will be, um, the prompt will keep the budget and like in mind, it will keep what we want, and then it'll also keep uh the strategy. It can either make its own strategy or the team that came up with the strategy, or like adapt the strategy, and it'll use a generic AI and it'll be building, it'll give you three things, the cost, the supply, and the plans. That's five things, bro. It also needs blueprints and code, bro, can you read? That's a part of the plans, bro, shut up, I'm getting to it, bro. Can't do a single thing right. Do it, bro, you talk then. Why don't you talk? No, you talk. Talk, bro. Come on. Dog, when are you gonna learn? How are you gonna... Exactly, just shut the hell up, bro. All right, we're back to it. All right, the plans need to have the blueprint, which will be the um which we can do 3D modeling of with like Nana Banana and LLMs, and it'll also have to create code for all the hardware that it said it'll use. So, like, based on the budget and based on the restrictions provided in the game manual, it'll be able to write the code. It'll give you step-by-step instructions, kind of like um the Lego instructions. It'll be able to provide step-by-steps, and it'll need to also get the supplies, not from anywhere, but like it'll need to get it from the Rev company which supplies all the um FTC um parts. You can use MCP or RAG to get the stuff from Rev, and it will use LLMs, it will use agentic AI to design the build and design the code, all that, and give it step-by-step. Building with... With CAD. And it should also... Move, oh my God! What are you right here? The torque thing, the physics, or the physics, bro. Okay, and they'll also need to calculate the physics and the math behind the hardware, how much RPM it needs, what the torque is, like based on the torque, weight, all that of the supplies. And it'll need to create like an optimal set of instructions, how many rotations per minute, all that good stuff. It'll need to verify with math, it'll also need to show the proof for the math, math. And this, this thing's goal is to save hours of FTC kits time, and it'll also be, it'll also be used as just like a starting point, but which will also turn into like the building guide for FTC students. Yeah, and then we also need, what word do we use? we need to give them like a agentic AI chatbot that they can talk to and describe their issues or whatever with, and it can help them come up with strategy and whatever questions they have. And we also need to build in a way for them to fill out forms and code email people for grants for their team. Basically, we're trying to streamline the entire FTC process. And we also want to build in something where it can optimize their piloting controls based on the pilot logs. For example, what buttons they're clicking, maybe we could rearrange them to be more optimal. And we're trying to help them in both the autonomous and piloted modes. So, for instance, helping them write code and again, make plans and whatnot. So just research this idea further.

Tab 3

FTC AI Copilot — Full Project Plan

Project Vision

Build an AI-powered FTC assistant that helps teams go from:

- game manual
- budget
- inventory
- strategy goals
- experience level

to a:

- legal robot concept
- complete parts list
- physics-verified mechanisms
- starter codebase
- build instructions
- CAD-ready blueprint
- optimized driver controls
- grant/sponsorship workflow

The goal is to save FTC teams hundreds of hours while making competitive robotics more accessible.

Core Problem

FTC teams struggle with:

- understanding the game manual
- deciding strategy
- choosing mechanisms
- staying within budget
- sourcing legal parts
- calculating torque/RPM
- writing code
- building efficiently

- managing outreach/sponsors
- training drivers

Most rookie teams waste weeks reinventing basic systems.

Solution Overview

“FTC Copilot”

An AI platform that turns team constraints into a buildable FTC robot plan.

The system combines:

- RAG (manual + FTC docs)
 - agentic AI workflows
 - CAD generation
 - physics calculations
 - FTC SDK code generation
 - strategy analysis
 - telemetry optimization
 - grant automation
-

Main Features

1. Rules + Strategy Engine

Inputs

Teams provide:

- FTC game manual
- budget
- inventory
- team skill level
- scoring priorities

- drivetrain preferences
- weight constraints
- end goals

AI Tasks

The AI:

- parses game rules
- extracts legal constraints
- identifies scoring opportunities
- suggests strategies
- detects inspection risks
- determines viable robot archetypes

Outputs

- strategy summary
 - recommended robot style
 - legal warnings
 - optimization suggestions
 - risk assessment
-

2. REV Parts + Budget Planner

Data Sources

Use:

- REV Robotics catalog
- CAD files
- BOM sheets
- product specs
- pricing
- availability

Future expansion:

- goBILDA

- AndyMark
- ServoCity

AI Tasks

The AI:

- selects compatible parts
- calculates total cost
- reuses inventory
- minimizes waste
- checks legality
- estimates assembly complexity

Outputs

- complete BOM
 - REV SKUs
 - pricing breakdown
 - missing components
 - optional upgrades
 - supplier links
-

3. Robot Design Generator

Goal

Generate a realistic FTC robot architecture.

Initial Robot Archetypes

Start with:

- tank drive
- mecanum drive
- intake systems
- claw systems
- linear slides
- arm systems

- hanging/endgame systems

AI Tasks

The AI:

- adapts dimensions
- chooses mechanisms
- balances weight
- places electronics
- generates subsystem layouts
- creates CAD-ready assemblies

Future Expansion

- Onshape integration
- Fusion 360 integration
- AI-generated geometry
- automatic STEP exports
- simulation environment

Outputs

- robot architecture diagram
 - subsystem layouts
 - CAD assemblies
 - blueprint visuals
 - exploded views
 - assembly references
-

4. Physics + Verification Engine

Purpose

Make the robot mathematically valid.

This is one of the strongest differentiators.

Calculations

The system calculates:

- wheel RPM
- drivetrain speed
- acceleration
- gear ratios
- arm torque
- slide load
- motor efficiency
- stall current
- battery draw
- center of gravity
- safety factor

Example Math

The AI should show:

- formulas
- assumptions
- derivations
- proof of calculations

Example:

- required arm torque
- available motor torque
- expected load
- recommended gear ratio

Outputs

- physics report
- mechanism verification
- overload warnings
- optimization suggestions

Example formulas:

Torque

$$\tau = rF$$

RPM to Linear Speed

$$v = r\omega$$

Gear Ratio

$$\text{Gear Ratio} = \frac{\text{Driven Gear Teeth}}{\text{Driver Gear Teeth}}$$

5. FTC Code Generator

Goal

Generate starter FTC SDK code automatically.

Supported Systems

- TeleOp
- Autonomous
- Sensor integration
- Odometry
- PID loops
- Driver assists

Libraries

Use:

- FTC SDK
- FTCLib
- Road Runner

AI Tasks

The AI:

- maps hardware
- generates Java classes

- creates autonomous skeletons
- writes subsystem logic
- configures controls

Outputs

- Java FTC project
 - hardware map
 - TeleOp code
 - autonomous routines
 - commented code explanations
-

6. LEGO-Style Build Instructions

Goal

Help teams physically assemble the robot.

Features

- step-by-step instructions
- numbered assembly process
- subsystem grouping
- visual references
- bolt/screw callouts
- wiring diagrams

Outputs

- assembly guide
 - printable instructions
 - PDF exports
 - interactive build mode
-

7. Driver Optimization System

Goal

Improve TeleOp performance.

Inputs

Collect:

- button presses
- joystick movement
- cycle timing
- failed scoring attempts
- operator conflicts

AI Tasks

The AI:

- identifies inefficient controls
- suggests remaps
- automates repetitive actions
- reduces driver workload
- improves cycle efficiency

Outputs

- control recommendations
 - macro suggestions
 - pilot analytics
 - efficiency scoring
-

8. FTC Team Operations Assistant

Goal

Help teams with non-robot work.

Features

- grant finder
- sponsorship outreach
- email drafting
- outreach tracking
- fundraising assistance
- budgeting help

Inputs

- team number
- location
- nonprofit/school status
- rookie/veteran status
- outreach goals

Outputs

- sponsor emails
 - grant applications
 - thank-you templates
 - fundraising plans
 - outreach reports
-

System Architecture

AI Stack

RAG Layer

Stores:

- FTC manuals
- Q&A forums
- REV docs
- SDK docs
- CAD references

Agentic AI Layer

Agents:

- strategy agent
- design agent
- code agent
- budgeting agent
- physics verifier
- operations assistant

CAD Layer

Possible integrations:

- Onshape API
- Fusion 360 API
- OpenSCAD
- STEP generation

Physics Layer

Use:

- custom formulas
- simulation libraries
- motion calculations
- drivetrain calculators

Frontend

Potential stack:

- React
- Next.js
- Tailwind
- Three.js for 3D viewing

Backend

Potential stack:

- Python FastAPI
- Node.js
- LangChain/LlamaIndex

- vector database
-

Recommended MVP

Start Smaller

DO NOT start with:

- full autonomous CAD generation
- complete simulation
- perfect robot generation

Start With

MVP Features

- upload game manual
 - enter budget + inventory
 - choose strategy
 - generate legal robot concept
 - create REV BOM
 - calculate torque/RPM
 - generate starter FTC code
 - create build steps
-

MVP Workflow

User Flow

Step 1

Upload:

- game manual
- inventory

- budget

Step 2

Choose:

- scoring priorities
- drivetrain type
- experience level

Step 3

AI Generates:

- robot concept
 - mechanism recommendations
 - BOM
 - physics report
 - starter code
 - build instructions
-

Differentiators

Why This Stands Out

Most FTC tools only do:

- CAD
- coding
- budgeting
- manuals

This combines ALL of them.

Biggest Competitive Advantage

The physics-backed explainability.

Not:

“Here’s a random robot.”

But:

“Here’s WHY this mechanism works mathematically within your constraints.”

Important Safety + Legal Notes

The AI should NEVER claim:

- “guaranteed legal”
- “competition-ready”
- “fully verified”

Instead say:

- “Based on the cited manual sections...”
 - “Verify with inspectors/Q&A forum.”
-

Long-Term Expansion

Future Features

- multi-agent autonomous planning
 - AI vision pipelines
 - automatic path planning
 - simulation sandbox
 - match strategy prediction
 - alliance optimization
 - scouting integration
 - real-time pit assistant
 - voice copilot
 - autonomous tuning
-

Suggested Hackathon Pitch

One-Line Pitch

“An AI-powered FTC copilot that transforms rules, budget, inventory, and strategy into a buildable robot plan with sourced parts, verified physics, starter code, and team management tools.”

Recommended Build Order

Phase 1 — Foundation

- RAG manual parser
- REV parts database
- BOM generator
- basic strategy engine

Phase 2 — Engineering

- physics engine
- drivetrain calculations
- torque verification
- starter code generation

Phase 3 — Design

- CAD integration
- blueprint generation
- LEGO-style instructions

Phase 4 — Intelligence

- pilot optimization
 - telemetry analysis
 - autonomous planning
 - sponsor/grant assistant
-

Best Tech Stack

AI

- OpenAI API
- LangChain
- LlamaIndex

Database

- Pinecone
- ChromaDB
- Supabase

Frontend

- React
- Next.js
- Tailwind
- Three.js

Backend

- FastAPI
- Node.js

CAD

- Onshape API
- Fusion 360 API

Final Goal

Create the first:

“End-to-End FTC Engineering Copilot”

A system that helps teams:

- design faster
- learn engineering
- reduce mistakes
- save money
- understand robotics
- compete more effectively

while still keeping students involved in the engineering process instead of replacing them.